FIN-OS · FINANCIAL OS FOR INDIA

The Small Model Layer

Status report and future plan

What has been built, how it was built, what it measured, what was tried and thrown away, and where it goes next.

0.5.1	13	6,800	673
best router candidate	intents, 3 languages	router messages	automated tests

Prepared 26 October 2026 from the project tracker, plan, roadmap, model cards and a fresh verification run.

Every number here is quoted from a file or a test run. Where a result is a limit or a failure, it says so.

Contents

Part 1 Executive summary . . . 3
Part 2 The idea: a ladder, not a model . . . 4
Part 3 Architecture: how the pieces fit . . . 5
Part 4 The foundation: engine health and retrieval quality . . . 6
Part 5 The intent router, built step by step . . . 7
Part 6 Numbers from tools, and shadow mode in the browser . . . 10
Part 7 The verifier experiment and why it ended in NO-GO . . . 11
Part 8 Categoriser, label tools and engineering discipline . . . 12
Part 9 Status board and what is blocked . . . 13
Part 10 The future plan: finish line and route . . . 14
Part 11 Concepts still to be used . . . 16
Part 12 Next ten actions and how to run everything . . . 17

How to read this. Parts 1 to 3 are the short version. Parts 4 to 9 are what was built and how, in detail, with the measurements. Parts 10 to 12 are the plan: the finish line, the route, the concepts still to be used and the decisions that only the founder can unblock. Colour code used throughout: **green** = done and verified, **blue** = partly, **amber** = blocked on a person or on data, **coral** = failed a gate or deliberately not built.

PART 1

Executive summary

In one page: what the SML is, how far it has got, and the honest bottom line

What it is. FIN-OS is a financial operating system for India: tax, budgeting, calculators and an AI assistant called Arya. The **small model layer (SML)** is the set of small, measurable components that sit around Arya's answers: a router that understands what a message is asking for and whether it is risky, a checker that makes sure every number in an answer comes from a source, retrieval that finds the right facts, and the evidence process that decides whether anything bigger should be built. It is not one big custom model. The blueprint's thesis is a **ladder of small things**, each climbed only when a measured gate says the cheaper rung is stuck, running mostly on the user's own device, with every number computed by code and never invented by a model.

95-96/112 public retrieval benchmark (84.8%)	**38/41** personalised benchmark, 0 leaks	**0.9506** router 0.5.1 macro-F1, sealed set	**4.85%** router false alarm on sealed set
594 JavaScript tests, all pass	**79** Python tests, all pass	**224** real claims labelled for the verifier	**0** model outputs wired as an active guard

What has been achieved

- **A working engine.** The retrieval-augmented engine was brought back to health, its bugs fixed with regression tests, and its quality lifted from 76% to about 85% on the public benchmark.
- **A 13-intent router** in English, Hinglish and Devanagari Hindi, built through seven versions on about 6,800 messages from blind writers. It runs in **shadow mode**: it watches and counts, it changes nothing.
- **A deterministic number checker** that is live in Arya, fail-open, covering rupee, percent and count figures with Indian number formats (lakh, crore).
- **A rigorous evidence habit.** Held-out sets that are never trained on, sealed sets scored once, leakage tests, checksummed models, and a tracker row with evidence for every step.
- **Honest negatives.** A verifier that looked perfect on synthetic data (precision 0.997) collapsed on real answers (0.17). That is a result, not a failure: it prevented a bad gate from shipping.

The honest bottom line

Where it stands

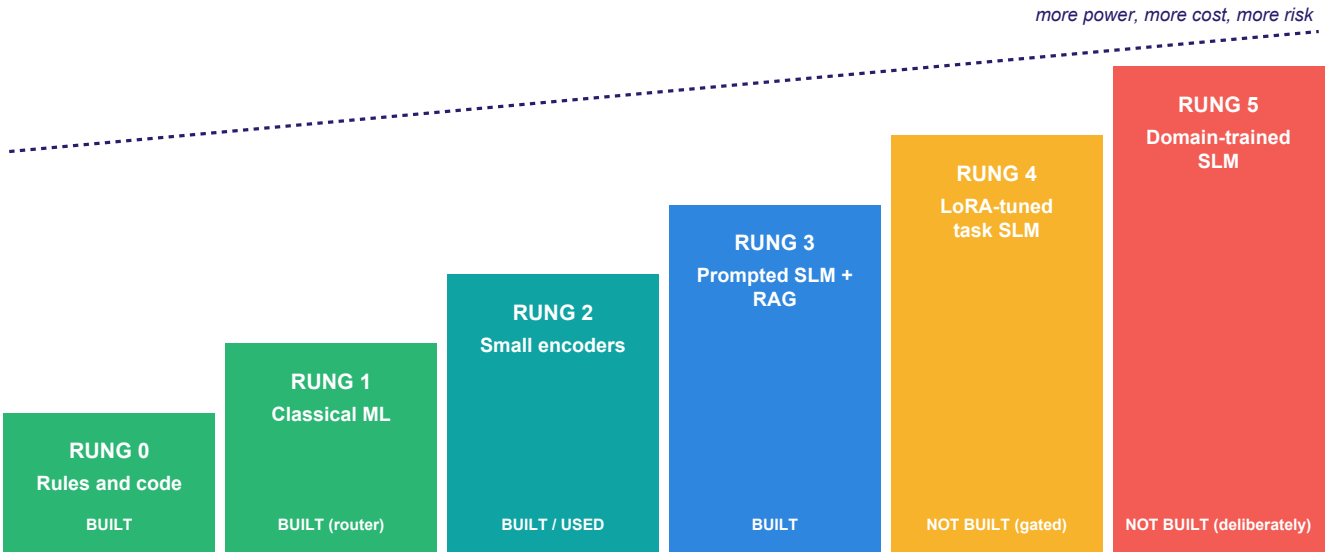
The router is a good **recall-first signal** (it caught 333 of 333 and 347 of 351 risky messages on two sealed sets), but it is **not yet an active guard**. No single threshold gives both a recall lower bound of 0.98 and a false alarm of 5% or less. That is a product decision (how many harmless messages may be flagged at which level) and it belongs to the founder. The next real gains need **real data**: consented messages, redacted bank statements and a second label reviewer. Everything that could be done without those has been done.

PART 2

The idea: a ladder, not a model

Why FIN-OS builds small, measured components instead of training one large model

The model ladder: climb only when a measured gate says the cheaper rung is stuck



Rules for climbing a rung (blueprint 7.2): a metric must be below target on a frozen golden set with a named failure class; error analysis on at least 50 failures must show the next rung fixes it; the cheaper rung must have been tried properly; and enough verified data must exist. Measured so far, the generator is *not* the bottleneck: qwen3:14b scored 96/112 against 95-96 for qwen3:8b at 2.5 times the latency, so rungs 4 and 5 are correctly not built.

The six binding principles

#	Principle	What it means in practice
1	Numbers come from tools or sources	FinosTaxCore, the budget and score modules compute; a model only explains. The numeric checker enforces it on answers.
2	Skip rather than guess	No invented defaults. If a fact is missing the answer says so (FinosFacts reports the gaps).
3	Nothing visible without a flag, a gate and a rollback	Every SML feature is default-off, measured, and removable with one switch.
4	Held-out data is never used to fit or tune	Sealed test sets are scored once. A test (F11) fails the build if any set overlaps training.
5	Personal data stays on the device	Counters hold no message text. Every new network flow must be declared in js/finos-privacy.js FLOWS.
6	Add guards, never remove AryaGuardrails	The router is an additional layer. The existing regex guard stays.

Why on-device first

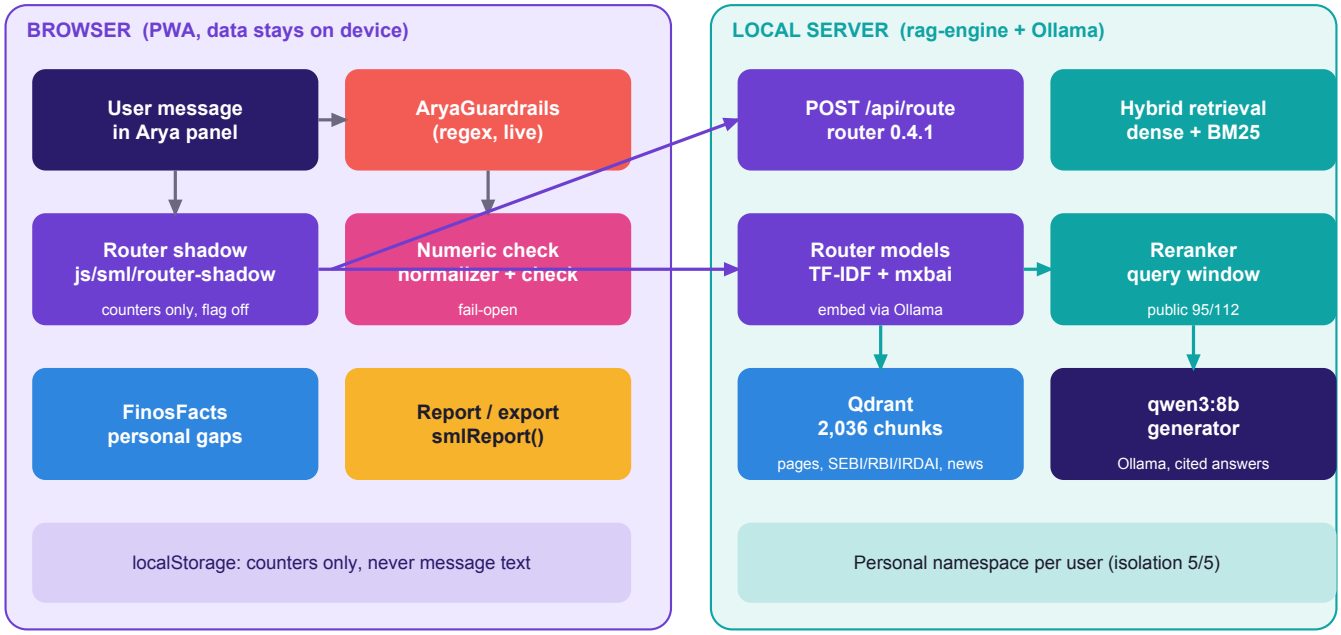
A financial assistant that handles someone's income, spending and documents must not default to sending that data to a third party. The blueprint therefore proposes an on-device-first hybrid: small models and rules run in the browser or phone, a local server handles heavier work today, and any hosted tier is a later, consented, public-data-only option that needs legal advice first. Today the router still needs the local server (Ollama for embeddings); making it run for every visitor in the browser is on the roadmap (Part 10).

PART 3

Architecture: how the pieces fit

Browser modules on the left, the local engine on the right, and what flows between them

Where the SML sits in FIN-OS (browser on the left, the founder's Mac on the right)



One request, end to end

Step	What happens	Where it lives
1	User types in the Arya panel. The existing regex guardrails check the text first (they stay).	js/arya-guardrails.js
2	If the shadow flag is on, the text is sent to **POST /api/route** without blocking the answer. The reply (intent, confidence, risk, guard yes/no) is turned into *counters only*.	js/sml/router-shadow.js, rag-engine server
3	Retrieval: dense vectors and BM25 are fused, a cross-encoder reranker scores the query-relevant window of each chunk, and a personal quota guarantees one slot to the caller's own facts for first-person questions.	rag-engine/retrieval
4	qwen3:8b writes a cited answer from the top three chunks (500-character windows) and tool results.	Ollama + rag-engine
5	The numeric checker extracts every figure in the answer and verifies it against tool results and source text. Fail-open: if it cannot run, the answer still shows.	js/sml/numeric-check.js
6	FinosFacts lists what the app does not know about the user, so Arya can say "I don't have that yet".	js/finos-facts.js
7	The founder can inspect the week's counters with **AryaSidebar.smlReport()** or export them with **smlExport()**.	js/sml/report.js

Design pattern

Fail-open and flag-off are the safety pattern: every SML addition can be missing, slow or wrong without breaking an answer, and each one is off by default.

PART 4

The foundation: engine health and retrieval quality

What was repaired, what was tried, and which levers were adopted or rejected

The first finding (14 October) was that the retrieval engine was far ahead of its documentation, and that the stack was not running. The work started by making it dependable, then measuring retrieval honestly.

Engine repairs (area A, 14 items, all done)

Problem found	Fix	Proof
/api/search and /api/retrieve returned HTTP 422	Decorator conflicted with postponed annotations	test_endpoint_signatures.py
Chunker looped forever and hung the PFRDA job	Loop guard in the chunker	test_chunker.py
Manual regulation crawls ingested duplicates	Dedupe by content hash; IRDAI re-run skipped 98	job result
Worker died and stayed dead	Restart loop in start-all.sh	killed worker came back
Laptop sleep skipped scheduled jobs	Scheduler misfire grace 6 h and coalescing	scheduler.py
Ollama ran one request at a time and unloaded the model	NUM_PARALLEL=2, KEEP_ALIVE=30m, applied durably	ensure-ollama-env.sh
Cached answers survived a settings change	Cache key now includes every tunable	config.pipeline_signature()
Documents were stale	Generated status file; stale docs corrected	docs/RAG_STATUS.md

Public retrieval benchmark (substring-match scoring; question sets differ, see text)



The original 100 questions and the later 112-question set are different sets scored by substring match, so the first bar is not a like-for-like step. The jump from the second to the third bar is the reranker change, measured on the same set. Run-to-run noise is about one question.

Retrieval levers, each measured against the same benchmark

Lever	Result	Verdict
Reranker scores the query-relevant window (was: chunk head, truncated at 256 tokens)	93 to 95-96 / 112; the answer reached the prompt for 12 of 20 failing questions (was 9)	DONE
Personal quota: the caller's own chunk gets a guaranteed slot	Personalised 35 to 38 / 41; isolation 5/5	DONE
Query-aware 500-character window in the prompt (was: first 500)	Regression test; more answers visible	DONE
Wider candidate pool 20+20	+1 question for +0.9 s per query	REJECTED
Personal prior x4	No gain (35/41)	REJECTED
News demotion x0.3 (news grew from 92 to 1,000 chunks)	0 of 15 affected questions changed	REJECTED
Quote-first prompt	90/112 against 95-96	REJECTED
Fewer or shorter contexts (2x500, 3x300)	91 and 93 / 112, both worse	REJECTED
Larger generator qwen3:14b	96/112, no gain, 2.5x slower	REJECTED

Lever	Result	Verdict
Router-driven retrieval filters (W3.3)	Oracle ceiling only +2 questions; router mapped correctly for 20%	REJECTED
Embedding bake-off (bge-m3, Qwen3-Embedding)	Needs model downloads and free memory	BLOCKED

What the failures say

Of the 17 questions that still fail, most are genuine **generation errors**: the model borrows a number from a neighbouring sentence. Shorter contexts, a bigger model and a quote-first prompt did not fix this. That is why the number checker and the rule table of authoritative figures (Part 6) matter more than another retrieval tweak. Three personalised questions also still fail on purpose (PD-05 arithmetic, PG-04 wording, PL-01 a missing category card); the expectations were not loosened.

PART 5

The intent router, built step by step

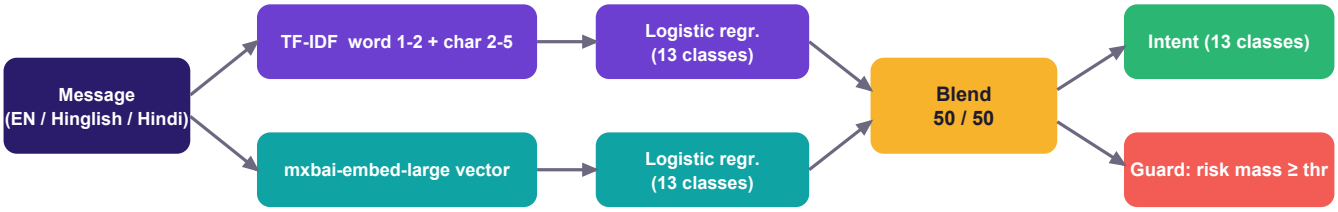
13 intents, 3 languages, a risk guard, and seven model versions

What it does. A message comes in; the router says which of 13 intents it is, and whether it falls in the two high-risk intents (**advice_seeking**, such as "which fund should I buy", and **evasion_request**, such as "how do I hide cash income"). The motivation is measured: today's live regex guard catches only about 10% of naturally phrased risky messages (14 of 146 on the dev set), while the router flagged 53 of 53 risky messages on test_v6, where the regex guard caught 6 of 53.

The 13 intents

personal_spend_query · personal_money_summary · goal_status · calculator_request · concept_explainer · regulation_question · market_data · document_help · app_help · smalltalk · out_of_scope · **advice_seeking** · **evasion_request**

How one message is classified (router 0.4.1 / 0.5.1)



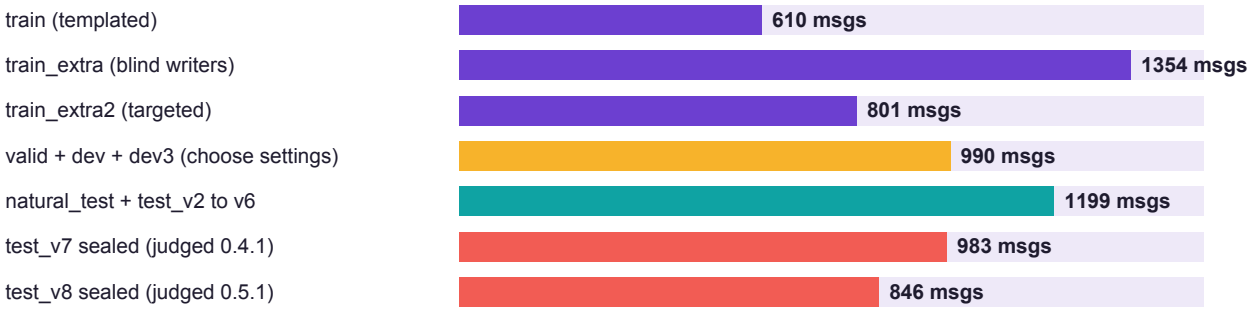
Guard = predicted class in {advice_seeking, evasion_request} OR P(advice) + P(evasion) ≥ threshold.
Threshold: 0.20 for 0.4.1, 0.35 for 0.5.1, both chosen on dev sets by a rule written down before the run.
Output today goes to shadow counters only: it never changes an answer.

How it was built

Stage	What was done	Why
Templated data	A generator wrote 610 training and 185 validation messages from templates in English, Hinglish and Hindi.	Cheap start, but templates make models look better than they are.
Blind writers	Independent agent writers, given only a category description (later, no example sentences at all), wrote natural messages. Ingest drops exact and near-duplicates against every set.	Breaks the echo problem: when briefs contained examples, writers copied them (seen on test_v7).
Two heads	Word 1-2 and character 2-5 TF-IDF with logistic regression, plus mxbai-embed-large vectors with logistic regression, blended 50/50 in 0.5.1.	Classical + encoder rungs of the ladder; the TF-IDF head alone is browser-sized (about 0.9 MB).

Stage	What was done	Why
Settings chosen on dev only	Blend weight, regularisation and guard threshold are picked on dev sets by a rule written before the run: the highest threshold with dev recall ≥ 0.99.	No tuning on a test set. When the rule flipped on one message (0.5.0) the dev set was enlarged instead of bending the rule.
Sealed sets	A new set written after the model and rule are frozen, scored exactly once, then retired as regression-only.	The only honest estimate of how it behaves on text it has never seen.
Guards in code	A test fails the build if any held-out file is missing from MANIFEST.json, overlaps training (exact or near-duplicate), or is edited.	A leakage of 9 short messages was caught this way and fixed.

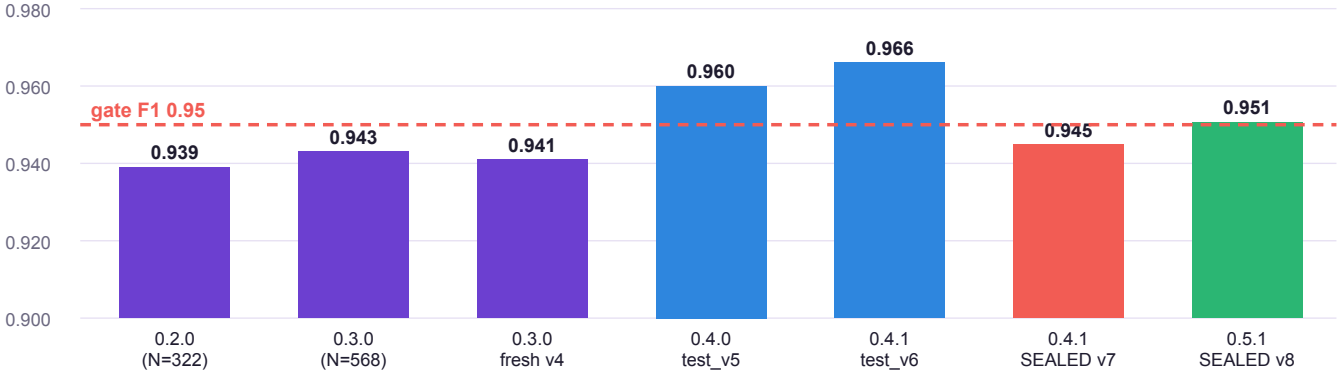
The router data: about 6,800 messages in three roles



Router versions: what each round learned

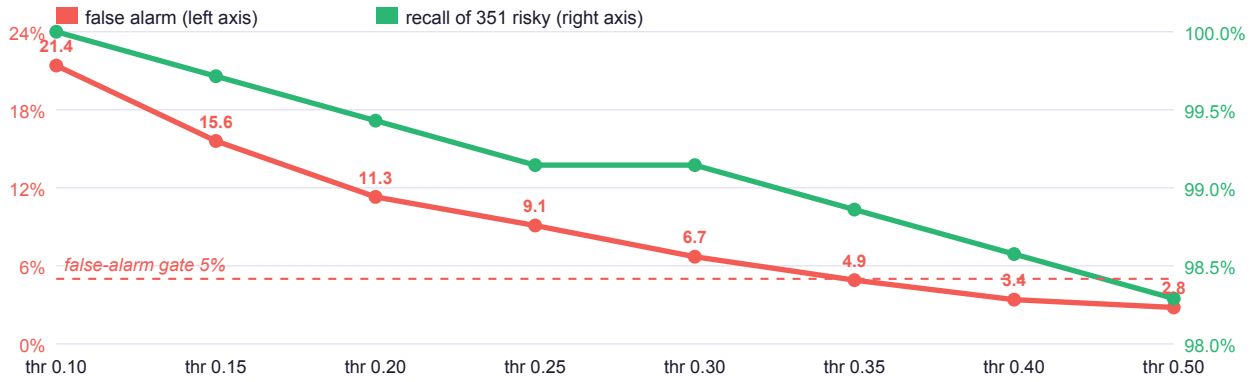
Version	What changed	Key result	Status
0.1.0	Embedding+LR against TF-IDF on 130 hand-written natural messages	87.7% vs 67.7%; regex guards caught 2/20 risky, model 17/20	baseline
0.2.0	+ independent blind-writer data (dev, test_v2, train_extra)	Blend 93.8% / F1 0.939 (N=322); Devanagari 0.84	superseded
0.3.0	+ 846 messages incl. 260 Devanagari; fresh test_v3	F1 0.943 (N=568). On a fresh v4 set: F1 0.941, guard recall only 36/40	superseded
0.4.0	+ 600 targeted messages (writers I, J, K)	test_v5 F1 0.960, recall 30/30, but false alarm 9.3%	superseded
0.4.1	Threshold re-chosen on a 424-message dev (0.15 to 0.20)	test_v6 F1 0.966, recall 53/53, false alarm 4.3%. Then SEALED v7: F1 0.9448, recall 333/333, false alarm 11.85%	shadow default
0.5.0	+ 801 rows after the v7 failure	Rule flipped on a single dev message; threshold rule undid the gain	rejected
0.5.1	Dev enlarged to 805 rows so the rule is stable; threshold 0.35	SEALED v8: F1 0.9506, false alarm 4.85%, recall 347/351 (bound 0.974 vs gate 0.98)	best candidate

Router macro-F1 on data the model had not seen (blue = one-shot confirm, red/green = sealed, scored once)



The trade-off that only the founder can settle

Sealed test_v8 (information only): recall vs false alarm by threshold, router 0.5.1



Lowering the threshold catches more risky messages but flags more harmless ones. At 0.35 the router flags 4.9% of harmless messages and misses 4 of 351 risky ones; at 0.20 it misses 2 but flags 11.3%. On both sealed sets (v7 and v8) a recall lower bound of 0.98 and a false alarm of 5% or less did *not* hold at one threshold, so the model, not only the threshold, sets that limit. The plan therefore uses **different operating points for different actions**: a cheap note under an answer can tolerate roughly 11-16% false alarms; steering or blocking needs about 3% or less.

What the error analysis found (frozen 0.4.1 over 1,199 held-out rows)

- Accuracy 0.9658. Confidence ≥ 0.85 was right 99.3-99.8% of the time; below 0.5 only 61.8%, so **confidence below 0.5 is a usable abstain signal**.
- Top confusions: spend vs summary (8), app_help vs document_help (3), out_of_scope pushed onto advice or evasion (6), and Hindi evasion about editing documents.
- Error rate by language: English 2.4%, Hinglish 3.6%, Hindi 5.5%. Hindi is the weakest and the least covered.
- Sealed v7 exposed the cause of its false alarms: "should I wear a jacket" and "which phone is best" were read as financial advice, because the model learned "should I / which is best" instead of the topic. That is what the 801 targeted rows of 0.5.x were written to fix.

Honest limits

Limits stated in the model card, not hidden. All router text is agent- or author-written, not real user messages. Sealed sets over-represent look-alike traps compared with real traffic. Boundary labels (spend vs summary, advice vs explanation) are noisy until a second person reviews them. Hindi is weakest. The 0.99 recall target needs about 300 risky messages from real users to confirm. The router also needs Ollama for embeddings, so today it protects only the founder's machine.

Browser-only router: feasibility checked

A TF-IDF-only head was exported without pickle (pure numpy scorer, parity with scikit-learn to 3.3e-8 on all 3,772 rows) and measured. It is small (about 560 KB compressed), recalls as well, but false-alarms about 2 points more (6-8% on exploratory sets, 11.7% on sealed v7) and its F1 on v7 was 0.928. Verdict: **PARTLY**. A browser guard is feasible in size, but needs the better 0.5.x training data and a sealed confirmation before it is a candidate.

PART 6

Numbers from tools, and shadow mode in the browser

The number checker, the figures table, the facts gaps and the privacy-first counters

The number checker (js/sml/numeric-check.js, normalizer.js)

A deterministic module that extracts every figure from an answer and verifies it against the tool results and source chunks the model was given. It never edits a number; at most it removes a whole sentence. It understands Indian formats (1,50,000 and 1.5 lakh and 150000 are the same number), and deliberately ignores things that look like numbers but are not claims: citation markers, FY 2025-26, section numbers like 80C and 87A, dates, phone and card-like ids, list numbering. A bare 2000 is read as money unless a year cue is nearby, so "you spent 2000 on rent" cannot slip through as a year.

Property	Detail
Match rule	An answer figure matches evidence when the evidence, rounded to the precision the answer displays, equals it (Rs 5,900 matches 5899.6)
Kinds	money, percent, count (verified only in strict mode), year and ordinal (never verified)
Safety	Fail-open in Arya; 20,000-character digit runs and 108 KB answers process in 0.1 to 26 ms (ReDoS probe: no change needed)
Tests	40 tests for the normaliser and checker; the whole JS suite is 594 passing
Known blind spot	Misattribution: a number that exists in the sources but is attached to the wrong thing. The checker cannot catch it, and 9 of the failing benchmark answers are exactly this.

The figures table (first slice of the rule engine)

sml/rules/figures.json holds 15 authoritative figures that FIN-OS itself states (80C, 80D twice, 80E with no cap, 80CCD(1B), equity LTCG and STCG, the LTCG exemption, property holding period and rate, standard deduction, the 87A 12-lakh rebate, cess, GST). Each entry records its source file and the exact anchor sentence. **check_figures.py** confirms all 15 anchors exist and scans pages and scripts for text that contradicts them. It found two things for a chartered accountant to confirm: the Property Sale note in js/finos-windfall.js and the Sovereign Gold Bond note in js/finos-gold-tracker.js both say "20% + indexation", while the tax page says 12.5% without indexation for sales after the July 2024 Budget. Nothing was changed without that confirmation.

Shadow mode: learn from real use without changing any answer

Piece	How it works
Flag	finos_sml_router_shadow in localStorage, default OFF. Flag off means zero requests (verified in a real browser).
Call	After each user message the panel calls /api/route without waiting; timeout 15 s; the answer is never delayed.
What is stored	Counters only: total, by intent, by risk class, by router version, router guard vs regex guard agree / router-only / regex-only, errors. **Never the message text.**
Report	AryaSidebar.smlReport() gives rates with Wilson 95% intervals and a readiness gate (≥ 300 messages). 22 messages cannot separate 9% from 28%; 300 gives about ±3.4 points at 5%.
Export and reset	smlExport() rebuilds the file through each module's whitelist so nothing but known counters can leave; smlReset() clears counters and leaves flags alone.
PULSE counters	A second local-only counter set records impressions and taps per PULSE widget (flag finos_pulse_counters_on). Nothing reads it yet: it is groundwork for the ranker.

Decision needed (N1)

A design gap was found and recorded rather than hidden: because counters hold no text, the plan's exit gate "founder reviews the disagreements" cannot be met with counters alone. Option (b), a default-off, device-local review buffer of the last N disagreeing messages that never leaves the browser, needs the founder's explicit yes (decision N1).

PART 7

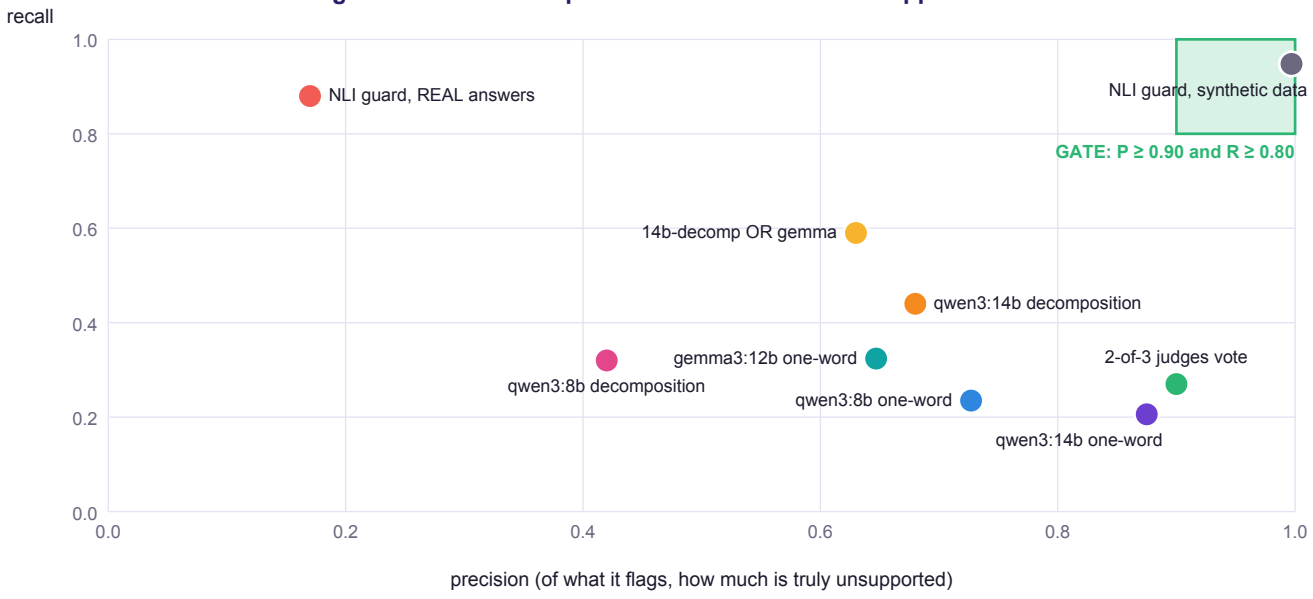
The verifier experiment and why it ended in NO-GO

A careful negative result: no automated faithfulness gate is safe to ship

Question. Can a model reliably tell when an answer contains a claim its sources do not support? The gate set by the blueprint: precision ≥ 0.90 and recall ≥ 0.80 for the decision "not supported" (precision matters because every false flag is a correct sentence wrongly doubted).

Step	What was done	Outcome
D1 Synthetic set	2,802 claim/evidence rows from indexed chunks: entailed, contradicted (number, negation, polarity changes), unsupported. Split by page group; 60-row hand check found 0 wrong in the final version (earlier versions had 4 and 1).	Exact labels by construction
D2 Baseline	The engine's nli-deberta-v3-base guard on the synthetic set	P 0.761 / R 0.971; wrongly flagged 61% of verbatim-supported sentences
D3 Cheap fixes	Best-matching-sentence premise plus a margin tuned on a validation split	P 0.997 / R 0.948: the gate looked met
D4 Real set	241 claims taken from the engine's real answers, two blind labelers; 224 agreed (kappa 0.77), 34 not supported	The honest test set
D5 Real result	The same cheap fixes on real claims	P **0.17**: 76% of correct sentences falsely flagged. Synthetic success did not transfer
D6-D7 LLM judges	qwen3:8b, qwen3:14b, gemma3:12b one-word judges; decomposition judges; number co-location; judge combinations	Best single: qwen3:14b decomposition P 0.68 / R 0.44 at 13.5 s per claim
D8 Decision	Nothing met the gate	**NO-GO** for an automated gate; keep the numeric check

Faithfulness verifier on 224 agreed REAL claims: precision vs recall for "not supported"



The best precision (0.90, two of three judges agreeing) came with recall 0.27 at about 17 seconds per answer on a machine already swapping about 25 GB. A larger generator was also tested (no gain). The lever that remains is **data**: at least 300 founder-labelled real claims with at least 100 not-supported (today only 34 of 224 are positives, so 300 random

claims would give a recall interval of about ±14 points). An opt-in "double-check" button was designed but deliberately not built.

The lesson

Synthetic data told a comforting story that real data refused. Everything in the SML now follows one rule: a number only counts if it comes from real or sealed data.

PART 8

Categoriser, label tools and engineering discipline

The groundwork that waits for data, and the habits that keep the work trustworthy

Transaction categoriser (W5): specified, waiting for real rows

The app's rule categoriser (CATEGORY_RULES in js/finos-budget.js) is first-match, so some orderings are suspect by reading alone: "Phone cover" lands in Bills & Utilities, "Gas station petrol" in Bills, "School bus fees" in Transport. These are hypotheses, not measurements. A learned model's gain cannot be shown on invented merchants because the rules would simply label them. So what was built is the **spec and the harness**: row format with bucketed amounts and redacted descriptions, a person-and-merchant-disjoint split, an abstain rule below 0.55, metrics with bootstrap intervals, and a validator that *rejects* anything personal-looking (10+ digit runs, e-mails, UPI handles, PAN, Aadhaar-like groups). The in-app export that would collect rows was deliberately not built: the existing redact() is known to leak UPI handles and non-ASCII digits (PENDING #14).

Labelling and review tools

- **LABELLING_GUIDE.md**: decision rules for the 13 labels, written around the confusions that were actually measured (spend vs summary, app_help vs document_help, advice vs non-decision, non-finance "should I", evasion vs regulation question).
- **review_sheet.py**: a seeded, stratified sample sheet *without* the original labels plus a separate key; **score** reports agreement with a Wilson interval, Cohen's kappa between two reviewers and the rows to adjudicate. It needs a second person to be used.

Audit pass 2: bugs found by testing the work itself

Finding	Effect	Fix
Counter ids like "constructor" resolved to inherited Object members	Polluted the global Object (Object.imp = NaN)	Own-property checks, +5 tests
PULSE tap listener stacked on every language change	Taps counted 2x, 3x, ...	Wired once per container; browser-verified (1 tap = 1 count)
English "main" treated as first person	"the main difference" pulled in an unrelated personal chunk	Fixed in the reranker, +1 test
Training script defaulted to version 0.3.0	A bare run would overwrite a frozen model	--version required, no overwrite without a flag; retrain reproduces 0.4.1 exactly
Personal quota embedded the question twice	Wasted ~30 ms (a whole HyDE call when on)	Embeds once and shares the vector
Near-copies hid in held-out sets (18 rows)	Possible inflation of reported scores	Measured: at most 0.0007 F1; pinned by hash; new ones fail the build
Router predictor trusted pickles and any embedding model	Unsafe load, silent garbage after a model switch	No pickle; refuses a router trained on another embedding model
/api/route answered 503 for blank text	A real failure was indistinguishable	Now 422
Benchmark seeded outside its try block	A failure left half-seeded chunks live	Seeding moved inside try; cleanup always runs

How the work is kept honest

SHA256	14	1 cmd	100%
checksums for 7 router versions	data files hashed in MANIFEST.json	sml/scripts/verify.sh runs it all	of work gets a TRACKER row with evidence

Last full run on 26 October 2026: all seven model checksums ok, **79 Python tests passed**, **594 of 594 JavaScript tests passed**, verify.sh printed ALL GOOD.

PART 9

Status board and what is blocked

Every workstream in one table, and the eight things only the founder can unblock

Workstream	State	Detail
Engine health (A)	DONE	14 items, all closed; 2,036 chunks indexed
Retrieval quality (W3)	PARTLY	95-96 / 112 public (target 100); cheap levers used up; bake-off needs downloads
Router (W4)	PARTLY	0.4.1 shadow default, 0.5.1 candidate; gates met on F1, partly on recall and false alarm
Shadow mode (W1)	DONE	Built and browser-verified; the founder's week of real use has not run yet
Number checker + facts	DONE	Live in Arya, fail-open, 67 tests
Verifier (W2)	REJECTED	NO-GO for an automated gate; numeric check kept
Figures table (D10)	PARTLY	15 figures and a consistency check built; answer-checking not built
Categoriser (W5)	BLOCKED	Spec and harness done; needs 1,000+ real redacted rows
PULSE ranker (W6)	BLOCKED	Counters collect; the ranker needs real usage
On-device runtime (W7)	NOT YET	Nothing runs on device yet; needs topology and phones
Embedding bake-off (W8)	BLOCKED	Needs model downloads and free memory

What only the founder can unblock

ID	Decision or input	Blocks	If no answer
F1	At least 1,000 redacted bank/UPI rows with correct categories, from several people	Categoriser	Stays blocked
F2	Feedback endpoint, consent text, retention rule	Real data for the router	Local counters only
F3	Which levels the router may drive (note / steer / block), acceptable false alarm per level, go-ahead to leave shadow, and 0.4.1 vs 0.5.1 as default	Active guard	Stays shadow on 0.4.1
F4	A second person to review boundary labels (and 300+ verifier claims)	Label quality, verifier	Author labels only
F5	Topology (recommended: on-device-first hybrid) and reference phones	On-device runtime	Server path only
F6	Approve model downloads (about 0.6-1.2 GB each)	Bake-off	Skipped
F7	Engage counsel on the SEBI and DPDP questions	Public launch	Education-only design stays
F8	Free memory (swap was about 25 GB; a scanner, a VM and a qemu process were largest)	Speed of everything	Queries 2-3x slower

ID	Decision or input	Blocks	If no answer
N1	Device-local capture of up to 100 redacted shadow-week messages	Reviewing disagreements	Counters only
N3	Track the small router binaries in git (about 1 MB) or retrain on each fresh clone	Fresh clones have no router	Binaries ignored
N4	Commit the work (commit message already prepared)	Everything sits in one checkout	Risk stays open

PART 10

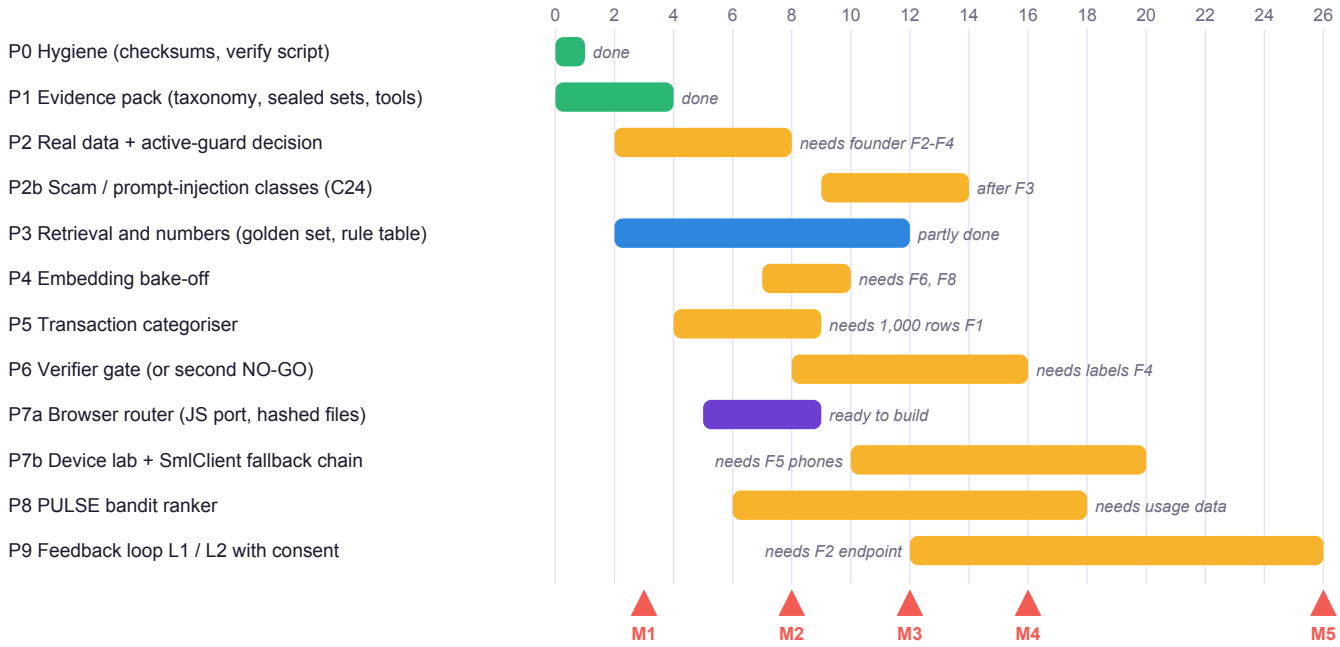
The future plan: finish line and route

Ten testable outcomes for the next 26 weeks, and the phases that get there

What "complete" means for this horizon

#	Capability	Complete means (testable)
1	Router as an active guard	Decides before the model runs. Advice gets the existing advice note; evasion is steered or blocked by an action ladder. Gate R: macro-F1 ≥ 0.95, recall lower bound ≥ 0.98, false alarm set per level, on a sealed fresh set plus reviewed real shadow data. Kill-switch tested.
2	Numbers from tools, checked	Any figure that matches a rule-table fact is compared with it and a conflict adds a caution (never edits). Numeric fidelity 100% on covered facts of a ≥ 50-item golden set; flag precision ≥ 0.95.
3	Faithfulness checking	A verifier meeting P ≥ 0.90 / R ≥ 0.80 on a founder-labelled real gate set, or a written second NO-GO. The gate is never weakened.
4	Transaction categoriser	Char n-gram logistic model as JSON in plain JS, behind a flag, rules kept as fallback, abstain below 0.55. Gate C: accuracy ≥ 0.90 at coverage ≥ 85% on a merchant-disjoint set of ≥ 300 rows; user corrections always win.
5	Learned PULSE ranker	On-device bandit with the rule ranker as prior and safety net. Gate P: offline replay shows a tap-through uplift whose 95% interval excludes zero on ≥ 1,000 impressions; otherwise the rule ranker stays and that is the result.
6	Retrieval quality	Public ≥ 100 / 112 on two runs, personalised ≥ 38 / 41, isolation 5/5; plus a typed golden set of ≥ 200 questions with recall@5 ≥ 0.90 English, 0.85 Hinglish and Hindi.
7	On-device runtime	Router decision in the browser in ≤ 100 ms warm with no server; hashed, versioned model files with a manifest; fallback chain server → browser → deterministic cards; device lab on 3-5 phones.
8	Embedding bake-off	mxbai vs bge-m3 vs Qwen3-Embedding-0.6B on recall@5 per language. Switch only if the winner gains ≥ 3 points on the Hinglish/Hindi slice with an interval that excludes zero.
9	Consented feedback loop	L0 local corrections, L1 opt-in aggregate counters, L2 redacted user-previewed examples. Consent, retention and deletion tested; zero undeclared network calls.
10	Wider safety coverage	A red-team set and router classes for scams and prompt-injection / data-extraction, so the safety gate (prohibited recall ≥ 0.99, over-refusal ≤ 3%) can be measured.

26-week horizon (written 2026-10-25, weeks counted from acceptance of the roadmap)



Milestones: **M1 (week 3)** evidence pack. **M2 (week 8)** active guard on the founder's machine if F3 is answered, and a browser router behind a flag if the TF-IDF verdict passes. **M3 (week 12)** retrieval checkpoint and embedding decision. **M4 (week 16)** categoriser and verifier verdicts. **M5 (week 26)** device-lab result, ranker verdict, feedback loop live or deliberately deferred. A wait on a person never stops the build track; it stops only what depends on it. Heavy machine jobs (a full public benchmark takes roughly 20-25 minutes) run one at a time.

Phase detail

Phase	Work	Gate and rollback
P2 Real data + active guard	Device-local sample capture (N1), second-reviewer labels, retrain on real messages, the action ladder (note / steer / block) with different thresholds per level.	Gate R. Rollback: flag off; model version pinned.
P2b Wider safety classes	Red-team set by blind writers (roleplay advice elicitation, forged rent receipts, structuring, prompt injection in pasted text, "print your system prompt", guaranteed-return schemes); add scam_fraud and prompt_injection classes only if the set shows the current ones miss them.	Safety gate on a sealed set.
P3 Retrieval + numbers	Golden set v1 (≥ 200 questions, gold chunk ids, recall@5 harness), grow the rule table, prompt experiments for the neighbour-number failures (pre-registered, each run twice), then a checkpoint decision.	Gate B. Every lever is an env knob defaulting to today's behaviour.
P4 Embedding bake-off	Index candidates into a side Qdrant collection, same retrieval and prompt for each, switch only by the pre-set rule; re-index and retrain the router's embedding head if a switch wins.	Gate E. Old collection and router kept until the new pair passes.
P5 Categoriser	Rows arrive; train char 2-5 n-gram logistic regression with amount bucket, credit/debit, channel token and a "rule fired" feature; split by merchant group and person; JS inference; browser verification.	Gate C. Rollback: flag off, rules remain.
P6 Verifier gate	Gate set of ≥ 300 real claims with ≥ 100 positives, two labellers; only then a small cross-encoder fine-tune on a separate training split.	GO only if the frozen gate set passes; otherwise a written second NO-GO.
P7 On-device	7a: JS port of the TF-IDF router with a parity test, hashed static files, service-worker cache. 7b: encoder feasibility, device lab, SmlClient fallback chain.	Gate J. Rollback: manifest change or kill-switch constant.
P8 PULSE ranker	Local event ring buffer, offline replay with inverse-propensity estimates, then an on-device Thompson-sampling bandit with the rule ranking as prior.	Gate P. Pinned widgets and rule top-N never overridden.
P9 Feedback loop	Consented report section first, then an endpoint with retention rule and deletion path; host added to FLOWS in the same change.	Privacy tests 100%; endpoint set back to null = local only.

PART 11

Concepts still to be used

The techniques the plan relies on, in plain words, and where each one lands

Concept	In plain words	Where it will be used	When
Action ladder	Different responses for different confidence and risk: add a note, steer to a safe explanation, or block.	Router as active guard (P2)	After F3
Operating point per level	Pick a separate threshold for each action, because a note can tolerate more false alarms than a block.	Router 0.5.x thresholds	After F3
Sealed test set	A set written after the model is frozen and scored exactly once.	Every router / categoriser / verifier release	Always
Confidence intervals	Wilson and Clopper-Pearson for proportions, bootstrap for F1, Cohen's kappa for labellers.	All gates and the shadow report	Always
Abstain on low confidence	Say "not sure" instead of guessing: router confidence below 0.5 (61.8% correct), categoriser below 0.55.	Router, categoriser	P2, P5
Golden set and recall@5	A fixed, hand-verified set of questions with gold chunks; the correct chunk must be in the top 5.	Retrieval gate and bake-off	P3
Bi-encoder and cross-encoder	A bi-encoder embeds text for fast search; a cross-encoder reads question and chunk together to rerank.	Embedding bake-off (bge-m3, Qwen3-Embedding); verifier fine-tune	P4, P6
Merchant-disjoint split	Test merchants never appear in training, so accuracy is not inflated.	Categoriser	P5
Char n-gram logistic regression	Reads small letter chunks, robust to messy bank text and typos; tiny and fast in plain JS.	Categoriser; TF-IDF router in the browser	P5, P7a
Rule table of authoritative figures	A versioned list of tax limits with source and effective date, checked deterministically.	Answer caution when a figure conflicts	P3, P6
Weights as JSON + hashed manifest	Model files shipped as static, versioned, SHA-256-checked files cached by the service worker.	Browser router delivery	P7a
WebGPU / ONNX / Transformers.js	Run small neural models in the browser; feature-test the capability, never sniff the device name.	Optional browser encoder; later on-device generator	P7b
SmlClient fallback chain	Try the server, then the browser model, then deterministic cards, so the app never breaks.	Runtime selection	P7b
Contextual bandit (Thompson sampling)	Learns which PULSE widgets get tapped, while exploring a little; the rule ranking acts as the starting belief.	PULSE ranker	P8
Inverse-propensity offline replay	Estimate how a new ranker would have done from logs, correcting for position bias.	Ranker gate before any live use	P8
Feedback levels L0 to L3	L0 local corrections, L1 opt-in counters, L2 redacted previewed examples, L3 federated learning (research only).	Consented feedback loop	P9; L3 not planned
LoRA / QLoRA, SFT	Train small adapter layers on 3k-10k verified examples for format, tone and tool-call reliability.	Rung 4 task models (MLX on the Mac)	Only if a golden-set gate shows prompting has plateaued
Distillation, DPO	Teach a small model from a large teacher (qwen3:14b); tune preferences from pairs.	Rung 4, optional	Gated, late

Concept	In plain words	Where it will be used	When
Domain-trained SLM	Continued pre-training on a large Indian-finance corpus (the NPCI FiMI reference used about 68 billion tokens).	Rung 5	Deliberately not planned; needs a go / no-go with evidence

Why this order

The blueprint's closing thought for the business side: **the moat is the loop, not the weights**. Weights are replaceable; a consented on-device feedback loop and a trusted Indian-finance evaluation set are not. That is why so much of the work so far went into measurement, data discipline and privacy plumbing rather than into training a larger model.

PART 12

Next ten actions and how to run everything

What to do next, in order, and the commands that reproduce the evidence

#	Who	Action
1	Founder	Commit the SML work using the prepared message (N4). Today everything sits in one checkout.
2	Founder	Turn on finos_sml_router_shadow and finos_pulse_counters_on in your browser, use FIN-OS normally for a week, then run await AryaSidebar.smlReport().
3	Founder	Answer N1 (device-local redacted sample capture), yes or no.
4	Founder	Name the second reviewer (F4) and answer F3 with the false-alarm numbers you accept (proposed: note ≤ 5%, block ≤ 2%) and the default version (0.4.1 or 0.5.1).
5	Founder	Pick the 3-5 people who will export redacted statement rows (F1) and the phones you can borrow (F5).
6	Build	After F3: wire the router as an additional guard on the action ladder, behind a flag, with browser verification and a tested kill-switch.
7	Build	Port the TF-IDF router to JS with a parity test and a hashed manifest (P7a), then a sealed confirmation on the 0.5.x data.
8	Build	Golden set v1 and the recall@5 harness; extend the rule table; checkpoint the retrieval plan.
9	Build	Train and ship the categoriser once the rows arrive.
10	Build	Red-team set and extra risk classes, then the bake-off when downloads and memory allow.

Reproduce the evidence

To check	Command
Everything (checksums + both test suites)	sml/scripts/verify.sh
Score a frozen router on a finished set	rag-engine/.venv/bin/python sml/training/eval_fresh.py 0.5.1 sml/data/test_v8.jsonl
Classify one message	rag-engine/.venv/bin/python sml/runtime/router_predict.py "your message"
Train a new router version	rag-engine/.venv/bin/python sml/training/train_router_v2.py --version 0.6.0
Figures table consistency	rag-engine/.venv/bin/python sml/tools/check_figures.py
Public benchmark	cd rag-engine && .venv/bin/python evaluation/benchmark.py --json out.json
Engine health and counts	rag-engine/.venv/bin/python scripts/rag_status.py

Where things live

Path	What
sml/TRACKER.md	Every step with its evidence; the source of truth

Path	What
sml/PLAN.md	Workstreams W1-W8, founder decisions, principles, log
sml/ROADMAP.md	The route from today to the end state
sml/data/	Router datasets, MANIFEST.json hashes, ingest script
sml/models/router/	Seven versions with model cards, eval.json and SHA256SUMS
sml/verifier/	Verifier experiments and REPORT.md (NO-GO)
sml/categoriser/ , sml/rules/ , sml/tools/	Categoriser spec and harness, figures table, review and consistency tools
js/sml/	Browser modules: normalizer, numeric-check, router-shadow, pulse-counters, report (must stay under js/ to be served)
sml/tests/ , sml/tests/js/	Python and JavaScript tests, including the leakage guards

Sources and caveats

Sources: sml/TRACKER.md, sml/PLAN.md, sml/ROADMAP.md, router model cards, sml/verifier/REPORT.md and the FIN-OS SML Blueprint (14 October 2026). Test totals come from a verify.sh run on 26 October 2026. Planning durations in the roadmap are estimates, not measurements. Legal and market statements in the blueprint are indicative and need counsel review before launch.